

Codebook (v2)

Supplemental material for “Debugging as Evidence-Driven Reasoning: Visualization Opportunities in Data-Intensive Programming” (IEEE VIS 2026 Short Papers).

This codebook contains seven categories. **Background/Context** and **Experience** are descriptive categories used for participant characterization (Table 1 in the paper); the remaining five — **Task-related Difficulty**, **Strategy**, **Information-Seeking**, **Attitude**, and **Need/Desire** — correspond to the five analytic themes reported in the paper. Examples shown are illustrative paraphrases rather than verbatim participant quotes (see the repository README for details on anonymization).

1. Background / Context

Definition: A description of the participant’s job role, task context, language, tools, and individual skill level, to help understand the context in which their debugging behavior takes place.

Coding Tip: Focus on “who is doing it, where it is being done, and what it is being done with.” Common signal words: *I work as...*, *I use MATLAB...*, *I debug in VS Code...*

No.	Subcode Name	Description	Illustrative Example
1	Job / Role	The participant’s professional identity, primary responsibility, or role in the project. Helps situate the work context in which debugging occurs.	“I’m a data analyst in an AI company.”

No.	Subcode Name	Description	Illustrative Example
2	Data Domain	The type or domain of data the participant works with (e.g., finance, healthcare, neuroscience, market analytics), reflecting the nature of the task and the complexity of the data.	“I work with neural activity recordings.”
3	Programming Language	The programming or scripting language the participant commonly uses, reflecting the debugging environment and skill propensity.	“I mainly use Python and R, but also write SQL.”
4	Environment / Tool	The tools, IDEs, platforms, or frameworks (e.g., VS Code, MATLAB, Jupyter, BigQuery) used by the participant for coding or debugging.	“I debug using VS Code and Google BigQuery.” / “Mostly use MATLAB toolboxes.”

No.	Subcode Name	Description	Illustrative Example
5	Workflow	The participant's overall workflow or sequence of operations, including cross-platform, cross-language, or multi-stage processing. Helps understand their macro logic when debugging the task.	"I validate SQL results and then link them with Apps Script automation." / "I run preprocessing locally and then push to the cluster."
6	Skill Level	The participant's self-assessed proficiency in the language or tool used (e.g., beginner / intermediate / expert), used to gauge differences in experience.	"Expert in MATLAB but beginner in C++."

2. Experience

Definition: The specific task or project context described by the participant that demonstrates their debugging or coding experience.

Coding Tip: Focus on the specific project or context being described. Common signal words: *In this project...*, *When I was fixing...*

No.	Subcode Name	Description	Illustrative Example
1	Data Processing Task	A data processing task performed during debugging or analysis (e.g., data cleaning, preprocessing) that typically involves manipulating the content, format, or structure of data.	“Running neural data preprocessing on cluster.”
2	Automation / Deployment	Tasks performed in an automated process or system deployment scenario, such as script automation, data pipeline scheduling, code go-live, or tool integration, intended to reduce manual repetitive steps.	“Write JavaScript in Google Tag Manager for event tracking.”

No.	Subcode Name	Description	Illustrative Example
3	Validation / Testing	Actions taken to confirm the correctness, completeness, or consistency of data or program results, typically through comparison, double-counting, or logical verification across methods, scripts, or data sources. The focus is on “validating” rather than “processing” data. Validation may occur at the code level (e.g., debugging function output) or the data level (e.g., SQL queries, cross-referencing datasets).	“Query customer emails to validate outputs.”

3. Task-related Difficulty

Definition: Objective difficulty of the task itself: data size, logic errors, performance bottlenecks, configuration issues, environment failures, etc.

Coding Tip: Check whether the sentence is describing “what the problem is.” Common signal words: *crash*, *not working*, *too large*, *mismatch*.

No.	Subcode Name	Description	Illustrative Example
1	Data / Performance Issue	Problems due to data size, resource limitations, or computational performance (e.g., slow execution, running out of memory, database overload, query failure).	“Database too large to run on AI tools.” / “Memory problem caused kernel crash.”
2	Logic / Implementation Error	Errors caused by code logic, function implementation, or dependency errors, including variable reference errors, path-configuration errors, and faulty conditional logic. The focus is on the implementation of the code itself, not the environment or data.	“Variable mismatch in function.” / “Wrong path configuration.”

No.	Subcode Name	Description	Illustrative Example
3	Environment / Configuration	Errors related to the development or runtime environment, such as missing packages, conflicting dependencies, incompatible versions, or kernel crashes. These problems usually relate to external configuration or system settings.	“Kernel crashed when running one cell.” / “Package missing.”
4	Existing Tool Limitation	A situation where an existing tool or platform cannot support the functionality required for a task — e.g., validation fails, an AI tool cannot handle large data, or there is a lack of interactivity. Reflects constraints on a tool’s functionality, contextual awareness, reliability, or ability to expose needed information.	“Column validation doesn’t work for complex schema.”

No.	Subcode Name	Description	Illustrative Example
5	Complexity / Scale	When code or a task is too large or complex, with many modules, leading to difficulty locating problems, high comprehension cost, or ambiguous debugging paths. Often reflects systemic challenges of the task itself.	“Codebase is huge and modularized, hard to locate errors.”
6	Knowledge Gap	The participant’s limited understanding of a language, tool, or debugging mechanism impedes interpretation of the problem, often requiring extra effort to make sense of errors.	“As a non-CS person, I have to put in extra effort to understand and fix it.” / “Sometimes the error messages aren’t very obvious.”

4. Strategy

Definition: The specific behaviors, methods, or approaches participants take to solve a problem.

Coding Tip: Focus on the “how.” Common signal words: *I tried...*, *I checked...*, *I asked...*

No.	Subcode Name	Description	Illustrative Example
1	AI Solution	Using an AI tool (e.g., GPT) to assist with debugging, interpreting, or generating code. Purposes may include (i) having the AI explain the cause of an error, (ii) generating or modifying code, and (iii) suggesting next steps. Captures all AI-based problem-solving activity.	“AI helps locate the problem.” / “Asked GPT to explain the error.” / “AI provides refined code and comments.”

No.	Subcode Name	Description	Illustrative Example
2	Manual Effort	The participant completes debugging steps through their own analysis or by hand — examining code line by line, observing variable values, manually modifying scripts, or reproducing the problem. This also includes code-level operations used to validate or confirm suspected errors, such as inserting print statements, adjusting parameters, and partial reruns. Encompasses inspection, hypothesis formation, controlled manipulation, and iterative re-execution; emphasizes human-led reasoning and experimentation.	“I manually checked variable types.” / “I looked at the functions being called and tested them outside the wrapper.” / “I ran some code to monitor memory usage.” / “Printed file path to check configuration.”

No.	Subcode Name	Description	Illustrative Example
3	Online / Social Support	Seeking help through external resources or social channels — searching online, reading forums, or asking colleagues / experts. Reflects socialized access to information.	“Searched online for solutions and explanations.” / “Asked peers or professionals for help.”
4	External Tool Assistance	Using built-in or external debugging tools (e.g., IDE debugger, Python toolbox, Spark UI) to visualize and trace program flow or monitor resource usage.	“I used the IDE debugger in Python or in VS Code.” / “The Spark Web UI lets me drill down into text-level details.”
5	Task Decomposition / Prompting	Breaking large or complex tasks into small, manageable steps, or progressively guiding the AI or oneself through refined questions (prompting). Reflects systematic thinking and task decomposition.	“Broke task into small subtasks for AI.” / “Prompted step by step.”

No.	Subcode Name	Description	Illustrative Example
6	Mental Confirmation	After experimenting or reviewing evidence, the participant verbalizes their understanding or causal reasoning — “what’s going on” and “why.” Reflects internal sense-making closure.	“After these steps, I already have an idea of what’s going on.” / “Now I know what caused it.”
7	Temporary Workaround / Code Bypass	Skipping or rewriting problematic code temporarily to keep progress when direct debugging is too time-consuming.	“If a section doesn’t work, I’ll comment it out and still run the rest.” / “I rewrote B to keep A→C running.”
8	Strategic Regulation / Goal Prioritization	Consciously regulating debugging effort — knowing when to stop, switch direction, or focus on execution first. Reflects meta-cognitive awareness of time and value.	“Knowing when to switch directions is very important.” / “First, my priority is to make the code run.”

5. Information-Seeking

Definition: The type of information a participant wants or tries to obtain during debugging in order to locate, understand, and solve problems.

Coding Tip: Check whether the participant is saying “I want to know something / I’m looking at something.”

No.	Subcode Name	Description	Illustrative Example
1	Program Output / API Response	The participant wants to see or understand program or API output to determine whether it runs correctly and whether the logic is as expected. Helps verify system behavior or identify bug locations.	“I wanted to check the API output.”
2	Log / Error Trace	Consulting system output information (e.g., logs, error messages, or stack traces) for clues to the cause of a failure. Usually involves reading system-generated messages (e.g., error message, kernel log).	“I read the kernel’s log file.” / “Looked at error message for clues.”
3	Variable / Object Info	Checking variable, object, or data-structure information at runtime to verify values, types, sizes, distributions, or missing entries. Reflects exploration of the program’s internal state.	“Checked whether it returns the correct data type or data structure.” / “I look at the distribution and check for outliers and missing values.”

No.	Subcode Name	Description	Illustrative Example
4	Online Information	Actively seeking relevant knowledge, explanations, or sample code from external resources (search engines, documentation, community forums) to understand or solve the issue.	“Found similar issue explanations online.”
5	System Resource	Focusing on runtime-environment information such as memory, CPU, GPU, or network usage, to determine performance bottlenecks or system state.	“Monitored memory usage while running code.”
6	AI-provided Answers	Receiving direct responses, explanations, or generated results from an AI tool (e.g., GPT) — text answers, debugging suggestions, or generated code — that help the participant understand the problem, validate hypotheses, or gain solution ideas.	“GPT explained that the merge failed because the join keys didn’t match.” / “AI generated a new code snippet that fixed the bug.” / “GPT suggested checking the variable type first.”

No.	Subcode Name	Description	Illustrative Example
7	Code Comments / Internal Documentation	Reliance on (or expectation of) comments, docstrings, or internal documentation to understand code logic, functionality, or execution flow. Unlike runtime output (e.g., logs), this is static, human-readable explanation written by developers; captures attention to comment quality and internal interpretability when reading unfamiliar code.	“Hopefully, the code has good comments so you can follow along.” / “I read through the docstrings to understand what each function does.”
8	Mental Clue / Hypothesis Formation	The participant forms clue-based hypotheses or inferences about the problem based on their own task, experience, and knowledge, establishing initial clues that are then verified through subsequent steps.	“That was the first clue.”

6. Attitude

Definition: Participants’ subjective views and feelings about debugging, tools (AI / visualization), and their own abilities.

Coding Tip: Look for emotional overtones (positive / negative). Distinguished from Task-related Difficulty: here it is “feelings,” not “facts.”

No.	Subcode Name	Description	Illustrative Example
1	AI-Related Attitude: Positive	Positive experiences with AI tools — effective at interpreting errors, generating code, or improving debugging efficiency.	“AI broke solutions step by step.” / “AI helped me understand the code better.”
2	AI-Related Attitude: Negative	Negative or skeptical views of AI tools — inaccurate results, limited functionality, or hard to trust.	“AI sometimes gives inaccurate results.” / “AI can’t connect live data.”
3	Visualization-Related Attitude: Positive	Found visualization helpful for understanding code logic, finding errors, or presenting results.	“Charts made the error pattern obvious.” / “Visualization helps when reviewing unfamiliar code.”
4	Visualization-Related Attitude: Negative	Found visualization irrelevant, of limited help, or intrusive to the task.	“Visualization isn’t necessary for simple tasks.” / “Diagrams distracted from the core issue.”
5	Self / Process Attitude: Positive	Rated themselves positively on debugging ability, efficiency, or learning growth.	“I’m becoming more efficient at debugging.”

No.	Subcode Name	Description	Illustrative Example
6	Self / Process Attitude: Negative	Expressed frustration, confusion, or anxiety about the debugging process.	“I feel lost when debugging complex systems.”
7	Neutral Feeling	A neutral or reflective viewpoint with no overtly positive or negative emotion. Often involves unused, context-dependent, or exploratory ideas. Spans the AI, Visualization, and Self categories.	“I’ve never used visualization tools.” / “It depends on the task.” / “I’m still learning how to use AI effectively.”

7. Need / Desire

Definition: A feature or form of support the participant explicitly or implicitly wants from a future tool, AI, or visualization system.

Coding Tip: Check whether the participant is saying “I wish / It would be nice if. . .” Needs may also be implied — e.g., “AI can’t connect to a database” implies a wish that it could.

No.	Subcode Name	Description	Illustrative Example
1	AI-Related Needs	Expectations of an AI tool’s functionality or intelligence, including more accurate error interpretation, contextual understanding, data access, or hinting capabilities.	“AI should give more detailed error explanations.” / “I wish AI could connect to local databases.” / “Would like AI to suggest next-step solutions.”
2	Visualization-Related Needs	Wishes for improvements to visualization support, including stronger visual correlation between data and code, display of variable changes, and process tracking. Reflects expectations of visualization usability in debugging.	“Want visualization charts explaining code execution.” / “Need variable-change visualization.”
3	Workflow / Integration Needs	Wishes for better integration of different tools or system features (e.g., AI and visualization) into a coherent debugging workflow. Focuses on “how tools work together” and “integration of debugging processes.”	“Prefer combined AI + visualization for debugging explanation.” / “Want AI and visualization to share context automatically.”

No.	Subcode Name	Description	Illustrative Example
4	UI / Interaction Needs	Wishes to improve the interface or interaction to make it more intuitive and efficient, including filtering, drag-and-drop, and interactive components.	“Want interactive UI filters in visualization.” / “Would like to click variables to trace their value changes.”

Theme-to-Finding Mapping

This table documents how the three cross-cutting findings reported in the paper were synthesized from the five analytic themes (T1–T5) derived during open, axial, and selective coding. Each finding draws on **multiple themes**, which is the basis on which it was identified as *cross-cutting* rather than tied to a single theme. Representative evidence is given by participant ID (P#); full code definitions appear in the codebook above.

Finding / Visualization Requirement	Contributing themes → code families	Representative evidence
F1 — Active Assembly of Fragmented Evidence → <i>Evidence Alignment</i>	T3 Information-Seeking (runtime state, log/error trace, external knowledge [AI + online], internal cues); T2 Strategies (external assistance: AI / online / tools; manual inspection across code, data, logs); T1 Challenges (Existing Tool Limitation: no single source suffices; unclear error messages force manual dataset inspection)	P6: unclear error messages force manual inspection of all datasets; participants iteratively cross-validate across runtime outputs, logs, and external explanations
F2 — Expected–Observed Discrepancy → <i>Expectation-grounded Comparison</i>	T3 Information-Seeking (internal cues: recognizing expected–observed mismatch, domain expectations; runtime state: types, ranges, distributions, missingness); T2 Strategies (mental modeling: expected-output comparison, hypothesis formation); T1 Challenges (Data/Performance Issue: deviating outputs, mismatched counts, misattribution; Logic Error: “result out of expectation”)	P4: “a rough idea of what the numbers should be... if something looks outrageous, something’s wrong”; P2: query returned 200 vs. an expected 207, revealing a channel misattribution

Finding / Visualization Requirement	Contributing themes → code families	Representative evidence
F3 — Limited State-Evolution Visibility → <i>State-evolution Tracing</i>	T5 Desired Support (step-wise data / variable evolution visualization); T2 Strategies (code experimentation: print statements, function isolation, intermediate-value checks, segmented execution; data-centric manual checking across stages); T1 Challenges (Complexity/Scale: hard to trace execution and inspect intermediate states; Environment/Configuration: cross-tool / pipeline boundaries); T3 Information-Seeking (variable/object info; execution traces)	P7: wants a flowchart showing a variable's value at beginning, middle, and end; P3/P4/P5: print statements and segmented execution to observe each step; P2: manual cross-database reconciliation across stages

Notes.

- *Background/Context* and *Experience* are descriptive categories used for participant characterization (Table 1) and are not among the five analytic themes (T1–T5) that underlie the findings.
- The *Attitude* theme (T4) primarily informs the discussion of mixed-mode presentation and AI as an auxiliary layer, rather than the three cross-cutting findings; it is therefore not a primary contributor in the table above.